北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 **姓名：**_____ **学号：**_____

考试时间：2026 年 10 月 12 日 **大班号：**_____**小班号：**_____

(第一次阶段测试) (大班号和小班号务必填写准确)

题号	2	3	4	5	6	7	8	**总分**
分数								
阅卷人								

北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、耳机、智能眼镜或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

注意事项：

1. 本试卷基本按照大班课程讲授次序分组，不是按照题目难度排序，注意合理安排时间。
2. 除特别说明外，默认技术前提是本课程课件和对应教材内容，例如：基于 x86-64 体系结构、Linux 操作系统、C 语言程序等；汇编程序是对应 C 程序主体功能实现，不做高级别优化，不一定是完整代码，采用 AT&T 格式。
3. 在试卷上指定位置答题，包括冒号后、等号后、箭头后、横线上、括号里等。

第 2 讲 共 8 分 得分：_____

2.1 位运算（2 分）

考虑以下 C 语言代码：

```
unsigned char x = 0xA6;
unsigned char a = x & 0x0F;
unsigned char b = x ^ 0x80;
```

求变量 a 和 b 的值，均用十六进制表示。

（1）a = _____（1 分）

（2）b = _____（1 分）

2.2 移位（2 分）

一个 8 位二进制数为 10110100。分别对其执行右移 2 位的操作，移出的位直接丢弃。

（1）采用逻辑右移，结果为：_____（1 分）

（2）采用算术右移，结果为：_____（1 分）

均用 8 位二进制表示。

2.3 整数（4 分）

考虑以下 C 语言代码，假设 int 和 unsigned int 均为 32 位：

```
unsigned int count = 3U;
int change = -5;

int a = (change < count);
unsigned int next = count + change;
int b = (next > count);
```

（1）a 的值为（十进制）：_____（2 分）

（2）b 的值为（十进制）：_____（2 分）

第 3 讲 共 16 分 得分：_____

3.1 IEEE 754 标准（4 分）

IEEE 754 单精度浮点数（float）采用 32 位编码，包含 1 位符号位、8 位阶码和 23 位小数域。现有一个 float 类型的数，其 32 位编码用十六进制表示为 0xC0E00000。

（1）按照 $v=(-1)^s \times M \times 2^E$，其阶码对应的实际指数 E 为：_____（2 分）

（2）该浮点数的十进制值为：_____（2 分）

3.2 IEEE 754 标准（4 分）

以下两个 32 位二进制编码按照 IEEE 754 单精度浮点数格式解释，分别表示什么？

（1）0xFF800000 → _____（2 分）

（2）0x7FC00000 → _____（2 分）

3.3 舍入规则（2 分）

按照 IEEE 754 默认的“向偶数舍入”规则，将以下两个二进制数保留小数点后两位。

（1）10.10100 → _____（1 分）

（2）11.01011 → _____（1 分）

3.4 浮点数格式（2 分）

某计算机采用与 IEEE 754 类似的 8 位浮点数格式：

- 最高 1 位为符号位 s；
- 随后 4 位为阶码 exp，偏置值 Bias=7；
- 最后 3 位为小数域 frac；
- 舍入采用“向偶数舍入”规则。

现有两个浮点数：

$a=1.100_2 \times 2^1$

$b=1.110_2 \times 2^1$

执行浮点乘法 a×b，乘积最终的 8 位浮点编码（用十六进制表示）为：_____（2 分）

3.5 浮点运算（4 分）

考虑以下 C 语言代码。假设 float 使用 IEEE 754 单精度格式，采用默认的向偶数舍入规则，每次浮点运算都按单精度舍入，不采用额外精度，也不改变表达式中的运算顺序。

```
float x = 16777216.0f;  // 2^24
float y = 1.0f;
```

```
float a = x + y;
float b = (x + y) - x;
float c = (x - x) + y;
```

回答以下问题：

（1）b 的值为：_____（2 分）

（2）c 的值为：_____（2 分）

第 4 讲 共 16 分 得分：_____

4.1 机器指令（2 分）

某程序经过编译、汇编和链接后，使用 objdump 反汇编得到以下代码：

```
400500: 48 89 d8        mov  %rbx, %rax
400503: 48 83 c0 05     add  $5, %rax
400507: c3              ret
```

（1）地址 0x400503 处的指令，其机器代码占用 _____ 字节。（1 分）

（2）以上三条指令的机器代码共占用 _____ 字节。（1 分）

4.2 寻址方式（2 分）

在 x86-64 计算机上，假设寄存器的内容为：

```
%rax = 0x600
%rbx = 0x1000
%rcx = 3
```

现执行：

```
movq  0x18(%rbx,%rcx,8), %rax
```

该指令访问数据的内存地址为（十六进制）：_____（2 分）

4.2 汇编程序（4 分）

以下是一段 x86-64 汇编代码。假设执行时的函数参数 x 存放在%rdi 中，函数返回值存放在%rax 中。

```
calc:
    leaq  (%rdi,%rdi,2), %rax
    salq  $2, %rax
    subq  %rdi, %rax
    ret
```

（1）执行第一条指令 leaq 后，%rax 中的值相当于：（ ）（2 分）

A. x B. 2x C. 3x D. 4x

（2）该函数的返回值相当于以下哪个 C 语言表达式？（ ）（2 分）

A. 7*x B. 11*x C. 12*x D. 13*x

4.3 汇编程序（4 分）

假设%rdi 的值为 0x3000，从该地址开始的 8 字节内存数据初始的值为 12。
执行以下代码：

```
movq    (%rdi), %rax
addq    $5, (%rdi)
subq    (%rdi), %rax
```

（1）执行结束后，%rax 中的值为（十进制）：_____（2 分）
（2）地址 0x3000 处的 8 字节内存数据，其值为（十进制）：_____（2 分）

4.4 汇编程序（4 分）

某 x86-64 计算机的寄存器初始状态为：

```
%rdi = 0x1000
%rsi = 2
```

内存中有两个 64 位整数，分别占据地址 0x1000 和 0x1008 开始的 8 个字节，其初始值分别为 7 和 11。
执行以下汇编代码：

```
    leaq    8(%rdi), %rcx
    movq    (%rcx), %rax
    leaq    (%rax,%rsi,4), %rdx
    subq    (%rdi), %rdx
    movq    %rdx, (%rcx)
```

回答：
（1）第二条指令执行后，%rax 中的值为（十进制）：_____（2 分）
（2）全部指令执行结束后，内存地址 0x1008 处的 64 位整数值为（十进制）：_____（2 分）

第 5 讲 共 16 分 得分：_____

5.1 条件码（2 分）

考虑以下汇编代码：

```
movq    $0x2A, %rax
testq   $0x08, %rax
```

执行后：
（1）条件码 ZF 的值为：_____（1 分）
（2）%rax 中的值为（十六进制）：_____（1 分）

5.2 条件跳转（4 分）

考虑下面的 C 语言代码：

```
long adjust(long x, long y)
{
    long t = x + 5;
    if (x > y)
        t = y - 2;
    return t;
}
```

假设 x、y 分别存放在%rdi 和%rsi，返回值存放在%rax。以下是一种对应的汇编实现，补全缺失的指令，并计算指定参数下的返回值。假设计算不发生有符号整数溢出。

```
adjust:
    leaq    5(%rdi), %rax
    ________%rsi, _______       (1)
    ____________________        (2)
    leaq    -2(%rsi), %rax
.Ldone:
    ret
```

（1）第（1）条指令是（整条指令写完整）：_____（2 分）

（2）第（2）条指令是：_____。（2 分）

5.3 循环结构（4 分）

考虑以下汇编代码。函数参数 x 存放在%rdi，返回值存放在%rax。

```
sum_pairs:
    movl    $0, %eax
    jmp     .Ltest
.Lbody:
    movq    %rdi, %rdx
    andq    $3, %rdx
    addq    %rdx, %rax
    shrq    $2, %rdi
.Ltest:
    testq   %rdi, %rdi
    jne     .Lbody
    ret
```

该代码与下面的 C 语言函数对应。补全缺失部分：

```
long sum_pairs(unsigned long x)
{
    long result = 0;
    while (_______________) {        // (a)
        result += ___________;       // (b)
```

```
        x = _________________;        // (c)
    }
    return result;
}
```

（1）补全(b)处的指令：_____（2 分）。

（2）当参数 x=0x1B 时，函数返回值为（十进制）：_____（2 分）

5.4 跳转表（6 分）

下面的汇编代码实现一个 C 语言函数 select(long x, long y)。参数 x、y 分别存放在%rdi 和%rsi 中，返回值存放在%rax 中。

```
select:
    cmpq    $5, %rdi
    ja      .Ldefault
    jmp     *.Ltab(,%rdi,8)

.L0:
    leaq    2(%rsi), %rax
    ret

.L1:
    leaq    (%rsi,%rsi), %rax
    jmp     .Lplus

.L2:
    movl    $1, %eax
.Lplus:
    addq    $3, %rax
    ret

.L45:
    leaq    -1(%rsi), %rax
    ret

.Ldefault:
    movl    $0, %eax
    ret
```

对应的跳转表为：

```
.section .rodata
.align 8
.Ltab:
    .quad   .L0
    .quad   .L1
    .quad   .L2
```

```
    .quad   .Ldefault
    .quad   .L45
    .quad   .L45
```

根据代码，计算以下函数调用的返回值（十进制表示）。

（1）select(1, 4) = _____（2 分）

（2）select(5, 4) = _____（2 分）

（3）select(-1, 4) = _____（2 分）

第 6 讲 共 16 分 得分：_____

6.1 栈操作（4 分）

假设 x86-64 计算机的栈指针%rsp 初始值为 0x3000，执行以下汇编代码：

```
movq    $0x35, %rax
pushq   %rax
movq    $0, %rax
popq    %rcx
```

（1）执行 pushq 指令后，%rsp 的值为：_____（2 分）

（2）全部指令执行结束后，%rcx 的值为：_____（2 分）

均用十六进制表示。

6.2 过程调用（4 分）

考虑以下 C 语言函数：

```
long twice(long a)
{
    return 2 * a;
}

long combine(long x, long y)
{
    long t = x;
    long v = twice(y);
    return t + v;
}
```

假设运算不发生有符号溢出。根据参数传递规则，前 6 个整数或指针参数依次通过%rdi、%rsi、%rdx、%rcx、%r8、%r9 传递，第 7 个及之后的参数通过栈传递。下面是一种对应的汇编实现，补全两条指令，每行 2 分。

```
combine:
    pushq   %rbx
    movq    %rdi, %rbx
```

```
    _____________________________   (1)
    call    twice

    _____________________________   (2)
    popq    %rbx
    ret
```

6.3 过程调用和栈帧（8 分）

考虑以下两个 C 语言函数：

```
long increase(long *p, long delta)
{
    long old = *p;
    *p = old + delta;
    return old;
}

long run(long x, long y)
{
    long local = 2 * x;
    long old = increase(&local, y);
    return old + local + x;
}
```

假设运算不发生有符号整数溢出。下面是一种实现函数 run 的汇编代码。参数 x、y 分别存放在%rdi 和%rsi 中，返回值存放在%rax 中。

```
run:
    pushq   %rbx
    subq    $16, %rsp
    movq    %rdi, %rbx
    leaq    (%rdi,%rdi), %rax
    movq    %rax, 8(%rsp)
    _____________________________   (1)//传递&local 的指令
    call    increase
    _____________________________   (2)//将更新后的 local 加入返回值的指令
    _____________________________   (3)//将原参数 x 加入返回值的指令
    addq    $16, %rsp
    popq    %rbx
    ret
```

（1）-（3）根据已有代码和注释，在代码中补全指令，每空 2 分。

（4）调用 run(4, 3)时，函数返回值为（十进制）：_____（2 分）

第 7 讲 共 14 分 得分：_____

7.1 数组（4 分）

考虑下面的 C 语言代码：

```
int m[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
int *row[3] = {m[2], m[0], m[1]};
```

假设二维数组 m 的起始地址为 0x3000。

（1）&m[2][1]的地址为（十六进制）：_____（2 分）

（2）表达式 row[0][1] + row[2][2]的值为(十进制)：_____（2 分）

7.2 结构体（8 分）

考虑以下 C 语言代码：

```
typedef struct {
    char tag;
    int v[3];
    long sum;
} Rec;

Rec a[3] = {
    {'A', {1, 2, 3}, 10},
    {'B', {4, 5, 6}, 20},
    {'C', {7, 8, 9}, 30}
};

Rec *p = &a[1];
int *q = p->v;
long result = p[1].sum + q[2];
```

假设结构体采用课件讲授的默认对齐规则，没有使用任何特殊的内存打包设置。数组 a 的起始地址为 0x1000。

回答以下问题：

（1）结构体 Rec 中的成员 v，相对于结构体起始地址的偏移量为：_____ 字节（2 分）

（2）sizeof(Rec)的值为：_____ 字节（2 分）

（3）&a[2].v[1]的地址为（十六进制）：_____（2 分）

（4）变量 result 最终的值为（十进制）：_____（2 分）

第 8 讲 共 14 分 得分：_____

8.1 结构与联合（3 分）

考虑以下类型定义：

```
struct S {
    char c;
    long x;
};

union U {
    char c;
    long x;
};
```

以下说法哪些正确？_____

（不定项选择，3 分。全对得 3 分，少选得 1 分，错选或不选得 0 分）

A. sizeof(struct S)的值为 16。

B. sizeof(union U)的值为 8。

C. struct S 的成员 c 和 x 占用相同的存储空间。

D. union U 的成员 c 和 x 具有相同的起始地址。

8.2 防御技术（3 分）

对于 x86-64 Linux 的程序，以下关于缓冲区溢出攻击及防御措施的说法，哪些正确？

_____

（不定项选择，3 分。全对得 3 分，少选得 1 分，错选或不选得 0 分）

A. “栈金丝雀”可以通过检查标记值是否改变，检测到部分缓冲区溢出。

B. ROP 攻击可以利用程序中已有的可执行代码，因此不可执行栈无法单独阻止此类攻击。

C. 栈地址随机化能够从根本上消除程序中的缓冲区越界写入。

D. 即使设置了不可执行栈，缓冲区溢出仍有可能修改保存在栈中的返回地址。

8.3 缓冲区攻击（8 分）

考虑以下 C 语言程序：

```
void read_input(void)
{
    char buf[16];
    gets(buf);  // gets()是本课程所讲授的有安全隐患的函数
}
```

```
void process(void)
{
    read_input();
    /* 后续操作 */
}
```

在一台 x86-64 Linux 计算机上，read_input 对应的汇编代码如下：

```
read_input:
    subq    $40, %rsp
    leaq    8(%rsp), %rdi
    call    gets
    addq    $40, %rsp
    ret
```

已知以下条件：

- 调用 read_input 时，call 指令的下一条指令地址为 0x4012A5。
- 刚进入 read_input 时，%rsp=0x1208。
- 程序未启用“栈金丝雀”。
- gets 函数读取输入的所有字符，并在末尾添加 \0 字符
- 假设下面讨论的越界写入，不会引起系统的其他异常情况处理操作。

回答以下问题：

（1） gets 执行时，数组 buf[0]的内存地址为（十六进制）：_____（2 分）

（2） 假设用户只输入连续的字符 A（ASCII 码为 0x41），随后按回车。若要求输入不会导致修改返回地址，最多可以输入多少个字符 A？_____（2 分）

（3） 按照第（2）问要求的情况，如果再多输入 1 个字符 A，那么 read_input 执行 ret 指令时，将跳转到哪个地址？用十六进制表示：_____（2 分）

（4） 如果程序启用“栈金丝雀”，是否可以阻止返回地址被修改？_____（2 分）